Final Project: Keystroke Prediction from sEMG Signals

Due: Monday, Dec 7, 2026, 11:59 PM PST

This final project is intended to give you an opportunity to apply the skills you've learned in the course to a practical research application, with an emphasis on exploration of architectures beyond CNNs. In this project, you may explore – in addition to CNNs – architectures like RNNs, CNNs+RNNs, Transformers, and other architectures we haven't explicitly covered in class (but can be easy to implement with `PyTorch`, `JAX`, etc.). You may use any deep learning frameworks to aid your implementation, including `PyTorch`, `PyTorch Lightning`, `TensorFlow`, `JAX`, etc. Our philosophy here is that you already understand the principles of training these networks, including backpropagation and optimization, in which case you are now well-equipped to use these packages.

Students may work in groups of up to (and including) 4 **people.** To find teammates, you may use the **Search for Teammates** functionality on Piazza. For more information, see `https://support.piazza.com/support/solutions/articles/48001158117-search-for-teammates`. You may also do the project individually. We provide **Google Cloud Education Credits** to support GPU computation for this project. Please see the Bruin Learn resources on this for instructions on how to claim and use credits.

Introduction

In class, we have worked extensively with computer vision data. In this project, we extend these concepts to sequential temporal data by exploring the task of predicting `QWERTY` keystrokes from surface electromyography (sEMG) signals. sEMG is a non-invasive technique that records electrical activity from muscles and holds promise for neural interfaces that support movement and communication, including for people with amputations or paralysis. To explore this task, we use the `emg2qwerty` dataset released by Meta Reality Labs, which contains synchronized sEMG recordings and ground-truth keystrokes from a `QWERTY` keyboard [2]. Further details about the dataset can be found in the accompanying paper: `https://arxiv.org/abs/2410.20081`.

Dataset

The `emg2qwerty` dataset is composed of preprocessed sEMG signals recorded from both wrists and the corresponding ground-truth key logger sequence. During each experiment,

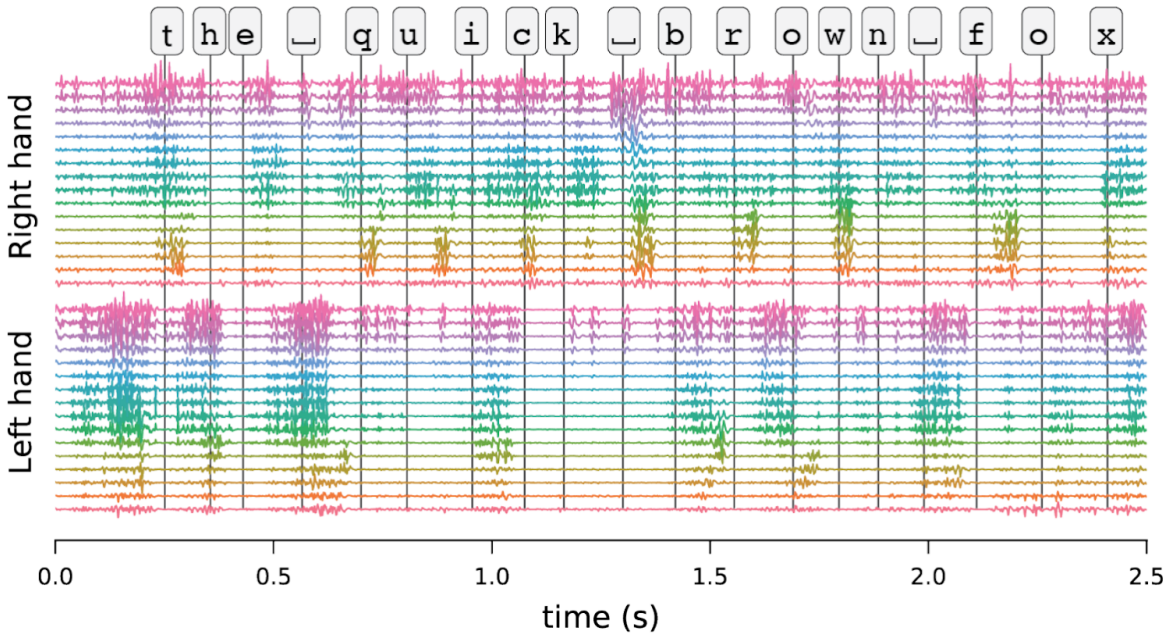


Figure 1: Visualization of *emg2qwerty* data, which includes sEMG signals from both hands and ground truth `QWERTY` key presses. [2]

a participant wore sEMG recording bands on their left and right wrists. Each band recorded from 16 sEMG electrode channels at a 2 kHz sampling rate (i.e., 2000 samples per second). Participants were then instructed to type while wearing the bands. The keys they struck, and the timestamps of when they were struck, were also recorded. This dataset therefore contains simultaneous sEMG signals from 32 total channels and corresponding key presses, illustrated in Figure 1. These data are released by Meta as hdf5 files. Using this data, it is therefore possible to **decode sEMG activity** to **predict what a person is typing**.

Although the full `emg2qwerty` dataset contains 108 users and 346 hours of recordings, processing the entire dataset may require substantial computational resources. **Therefore, the baseline project only requires training on a single subject (ID #89335547), whose data can be downloaded from** `https://ucla.box.com/s/3xc4nwpfjfpo6ydjs94t0v2kuq37d5eg`**.** Students can earn maximum credit by conducting thorough experiments on this subject alone, but are welcome to explore additional subjects or the entire dataset. The entire dataset can be accessed through the official GitHub repository: `https://github.com/facebookresearch/emg2qwerty`. Note that each `hdf5` file corresponds to a **single experimental session**.

Task inputs and outputs

The primary objective is to correctly predict typed keystrokes given a sequence of sEMG signals. You will therefore be designing a deep learning pipeline that takes input sEMG signals $x \in \mathbb{R}^{T_{\text{in}} \times N \times B \times C}$ and predicts an output character probability sequence $\hat{y} \in \mathbb{R}^{T_{\text{out}} \times N \times \text{num_classes}}$. Here, T_{in} is the temporal length of the input sEMG signal, N is the batch size, B is the number of EMG recording bands ($B = 2$), C is the number of electrode channels in each band ($C = 16$), and T_{out} is the temporal length of the model output. The output character sequence may contain both typed characters, as well as the `null` character (indicating nothing was typed at that time).

The ground-truth labels $y \in \mathbb{R}^{T \times N}$ represent the *class indices* for the T non-`null` characters present within the input time window T_{in}. Because in general, $T \neq T_{\text{out}}$, this is not a simple classification task. Hence, we recommend using the **CTCLoss** [1], a specialized loss function for situations like this. We provide some background below.

CTC Loss

Proposed in 2006 [1], the **Connectionist Temporal Classification** (CTC) loss aims to address classification under settings with **unsegmented sequential data**. This refers to situations like ours where the labeled sequence consists of characters typed at *arbitrary timestamps*. For comparison, in a segmented setup, we would instead create uniform time intervals and provide a ground truth character for each time segment, which is unideal for our task. The CTC loss is already implemented in the codebase; however, we encourage you to read more in-depth about it for a better understanding (e.g., `https://distill.pub/2017/ctc/`).

Model Evaluation

Your models are to be evaluated using the **Character Error Rate (CER)** metric. The CER computes:

$$\text{CER} = \frac{S + D + I}{N_{\text{char}}},$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N_{char} is the total number of characters.

Training and evaluating a baseline model

The GitHub repository containing the implementation of the baseline model can be found at `https://github.com/Calvin-Pang/emg2qwerty`. We also provide an initial setup

notebook for `Google Colab` in this repository. To train the base TDS model (see [2]) on a single subject of data, please follow the notebook `Colab_setup.ipynb`. In our experiments, the training on a single subject (with the most amount of data) took approximately 1 hour on `Google Colab` for 40 epochs and achieved a validation CER of 30%.

Project Requirements and Suggested Directions

Each project must complete the required task (a) and explore at least one additional direction from (b)–(f). You are also welcome to propose your own experimental direction beyond those listed below.

(a) **Required: Architecture Comparison.** Train, evaluate, and compare different neural network architectures on the provided single subject, ID #89335547. You must experiment with at least one recurrent architecture (e.g., RNN, LSTM, or GRU) and compare it against the provided baseline. Exploring additional architectures, such as CNN+RNN hybrids or Transformers, is encouraged.

Please note that we have already split the data for subject #89335547 into train/val/test splits at `https://github.com/Calvin-Pang/emg2qwerty/blob/main/config/user/single_user.yaml`. Please use these splits for the project. Use the validation set for model and hyperparameter selection and the test set for final evaluation.

(b) **Data Preprocessing and Augmentation.** Experiment with different data preprocessing or augmentation techniques and investigate their effects on CER.

(c) **Channel Ablation.** How many channels are needed to achieve good decoding performance? Investigate the relationship between the number of electrode channels and CER.

(d) **Training Data Efficiency.** How much training data is needed to achieve good decoding performance? Investigate the relationship between the amount of training data and CER.

(e) **Sampling Rate.** How fast does sEMG data need to be sampled for good performance? Investigate the relationship between sampling rate and CER.

(f) **Further Exploration.** You may propose alternative research directions, provided that they include meaningful experiments and analysis comparable in scope to (b)–(e).

We encourage meaningful experimental comparisons and analysis of why different approaches succeed or fail. The quality of your experiments and insights is more important than simply the number of models or methods evaluated.

Project Logistics

Submittables

Each group should submit a write-up of their project work, exceeding no more than 7 pages. The Contribution Statement, AI Usage Statement, and References are excluded from this page count. It is fine to be below the page limit; this is the *maximum*. We will also ask you to submit your code, so that we can validate your results.

The writeup must adhere to the following template: `https://media.neurips.cc/Conferences/NeurIPS2024/Styles.zip` – so that we can judge all writeups in the same manner without having to worry about different font sizes, etc. To remove the line numbers, modify the following line at the beginning of the `.tex` file to include the `final` option: `\usepackage[final]{neurips_2024}`.

Writeup

In the write-up, there should be the following sections:

(a) **Abstract** – A brief description of what you did in the project and the results observed.

(b) **Introduction** – Do not use the introduction to formulate the general problem of sEMG decoding, as we are all familiar with the problem. Instead, use the introduction to set up and motivate the research questions and techniques you pursued. For example, if you focused on minimizing CER in a single subject, or maximizing generalization performance using minimal amounts of data, you may motivate why this is important.

At the end of the introduction, include a brief summary of your work, preferably presented as bullet points. These should highlight your key approaches, experiments, and findings, rather than simply listing the methods you implemented.

(c) **Methods** – State the methods of your project (such as the architectures, data preprocessing, or other techniques used).

(d) **Results** – State the results of your experiments.

(e) **Discussion** – Discuss insights gained from your project, e.g., what resulted in good performance, and any hypotheses for why this might be the case.

(f) **Contribution Statement** – Clearly describe the contributions of each group member, including their responsibilities in model implementation, experimental design, analysis, and report preparation. All group members are expected to actively participate in the project and understand the submitted work.

(g) **AI Usage Statement** – Clearly disclose whether and how AI-assisted tools (e.g., ChatGPT, Claude, Gemini, etc) were used during the project. If applicable, specify the tools used and their roles in the project. If no AI-assisted tools were used, explicitly state so.

(h) **References** – List references used in your writeup.

Grading

Here we outline the criterion by which we will grade the project. Concretely, the final project will be graded on a scale of 20 points, but each section is assigned points so that the sum total can exceed 20 points. Your final project score will be capped at 20 points. You should aim to do a good job in all areas.

(a) **Creativity (7 points)**

- How creative and/or diverse is the approach taken by the student(s)?
- Do the student(s) implement and try various architectures and algorithms?
- Are multiple directions explored?

(b) **Insight (7 points)**

- Does the project reveal some insight about why approaches work or did not work?
- Is there reasonable insight, explanation, or intuition into the results? (i.e. you should not just blindly apply different algorithms to a problem and compare them.)

(c) **Performance (6 points)**

- Does the project achieve relatively good performance on the problem?
- How do different algorithms compare?

(d) **Write-up (4 points)**

- Are the approach, insight, and results clearly presented and explained?

Dissemination of work is an important component to any project.

Use of AI Tools

Students may use AI-assisted tools for debugging, writing assistance (e.g., language editing and polishing), etc, subject to course policies. However, all students are responsible for understanding, validating, and explaining the methods, experiments, and results presented in their submissions. The course staff reserves the right to request additional details

or verification of submitted work, such as an individual oral examination to assess their understanding and contributions to the project.

References

[1] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural 'networks. volume 2006, pages 369–376, 01 2006.

[2] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I Mandel. emg2qwerty: A large dataset with baselines for touch typing using surface electromyography, 2024.